Assignment: Using KNN to classify Iris dataset

1. Introduction

This assignment is developed to use a K-Nearest Neighbors (KNN) classifier to accurately identify three species of the Iris flower (*Setosa*, *Versicolor*, and *Virginica*) based on four physical measurements: sepal length, sepal width, petal length, and petal width.

2. Conceptual Overview: What is KNN?

K-Nearest Neighbors (KNN) is a non-parametric, supervised learning algorithm used for classification. Its fundamental logic is based on feature similarity; it assumes that similar data points exist in close proximity to each other within a multi-dimensional space**.**

Unlike algorithms that build a complex mathematical model (like Linear Regression), KNN is a "Lazy Learner." It simply stores the training dataset and waits until a prediction is needed.

- When a new data point (a test Iris flower) is introduced, the algorithm calculates the distance between this point and all existing points in the training set.
- It selects the K closest points (neighbors).
- The new point is assigned the class that is most common among its K neighbors through a majority vote.
- Distance Metric: To determine "closeness," we typically use Euclidean Distance, which calculates the straight-line distance between two points (p and q) in n-dimensional space:

$$d(p, q) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2}$$

- The value of K determines the "smoothness" of the model's decision-making:
  - Small K (e.g., K=1): The model is very sensitive to noise or "weird" data points. This can lead to Overfitting.
  - Large K (e.g., K=50): The model becomes too "blunt" and ignores local patterns. This can lead to Underfitting.
  - The Sweet Spot: Usually found using Cross-Validation (testing different values to see which performs best on unseen data).

3. Methodology & Code Explanation

- **Data Acquisition and Preprocessing:**
  - Iris Dataset from UCI is used, which contains 150 samples across three species.
  - Feature Scaling (StandardScaler): KNN calculates the Euclidean distance between points. If one feature (like length) has a larger range than another (like width), it will unfairly dominate the "distance." We scale the data so all features have a mean of 0 and a standard deviation of 1.

- Stratified Splitting: We use a 70/30 split with stratify=y. This ensures that the 70% training set and 30% test set both contain exactly the same proportion of each flower species, preventing "sampling bias."

- **Hyperparameter Tuning (Finding 'K'):** Instead of guessing a value for K, we perform a Grid Search combined with 5-Fold Cross-Validation.
  - We test K values from 1 to 20.
  - Cross-Validation splits the training data into 5 parts, training on 4 and validating on 1. This ensures the chosen K is stable and not just lucky on one specific shuffle.
- **Model Training:** The final model uses the "Optimal K" found in the previous step. We also implement weights='distance', which gives closer neighbors more "influence" than further ones, making the model more robust to outliers.

4. Performance Evaluation

To verify the model's reliability, we use three primary metrics:

1. Accuracy Score: The overall percentage of correct predictions.
2. Confusion Matrix: A 3 x 3 grid showing exactly where the model might be confusing similar species (like *Versicolor* vs. *Virginica*).
3. Classification Report:
   - Precision: How many predicted positives were correct
   - Recall: How many actual positives were found
   - F1-Score: The harmonic mean of Precision and Recall.

5. **Summary of Results**

```
Optimal K Found: 14
Final Test Accuracy: 0.9556

Confusion Matrix:
[[15  0  0]
 [ 0 15  0]
 [ 0  2 13]]

Detailed Report:
                 precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        15
Iris-versicolor       0.88      1.00      0.94        15
 Iris-virginica       1.00      0.87      0.93        15

       accuracy                           0.96        45
      macro avg       0.96      0.96      0.96        45
   weighted avg       0.96      0.96      0.96        45
```

Figure 1: Code Output

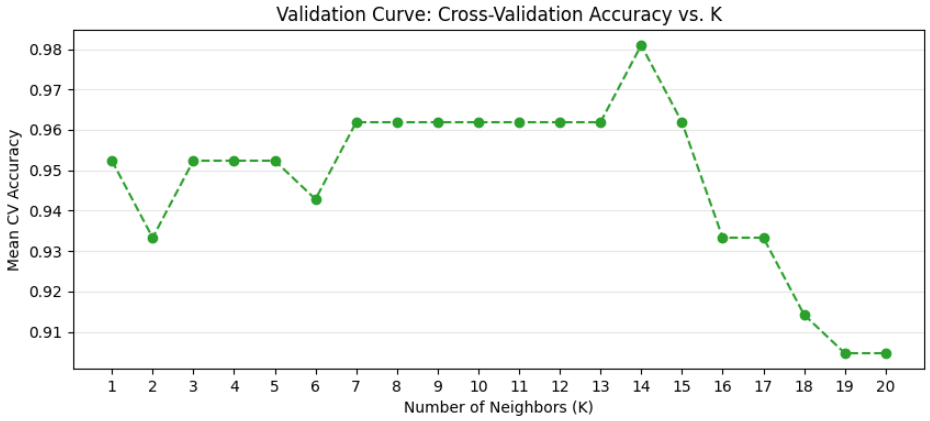

Figure 2: Validation Curve

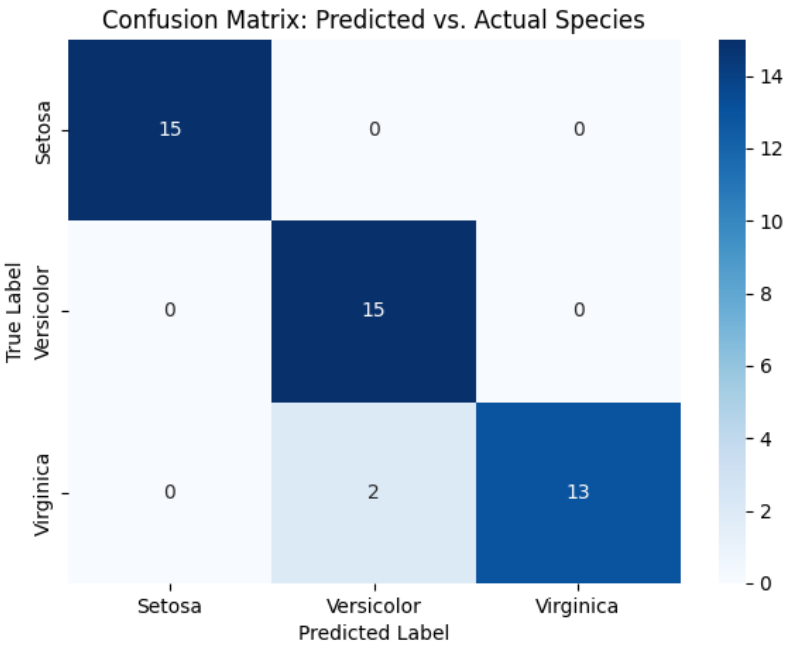

Figure 3: Confusion Matrix

-Avirup Roy
[232010007010]
CSE, 6th sem